

Índice

1. Descripción de la Entidad	1
2. Organigrama y Ubicación del Alumno	1
3. Descripción de Tareas Realizadas (Orden cronológico)	2
3.1. Configuración del Entorno de Desarrollo con Docker	2
3.2. Análisis de Migración de Qiskit v0.46 a v1.0	2
3.3. Implementación y Análisis del Sistema RAG	4
3.4. Verificación y optimización del RAG	5
3.5. Escritura del paper	6
4. Tecnología Empleada	6
5. Problemas Identificados a lo largo de las Practicas	6
6. Evaluación de la Formación Académica y Desempeño Profesional	7
6.1. Desempeño como Futuro Profesional	7
6.2. Fortalezas y Debilidades de la Formación Recibida	7
6.3. Adaptación Personal y Relaciones Interpersonales	8
7. Conclusión	8
8. Bibliografía	9

1. Descripción de la Entidad

La Práctica Profesional Supervisada (PPS) se desarrolló en el **Laboratorio de Investigación y Formación en Informática Avanzada (LIFIA)**, unidad de la Facultad de Informática de la UNLP.

Actividad y Posicionamiento: El LIFIA es un centro de referencia en Ingeniería de Software y Ciencias de la Computación. Sus actividades principales comprenden la generación de conocimiento científico de alto impacto y la transferencia tecnológica al sector público y privado.

Contexto del Proyecto: Las tareas se enmarcaron en un proyecto de I+D orientado a la publicación de un *extended abstract* sobre computación cuántica. La iniciativa responde a la rápida obsolescencia de las herramientas del dominio, lo cual exige frecuentes migraciones de código. Esta carga técnica representa una barrera significativa para los físicos experimentales, el perfil predominante en el área, desviándolos de su investigación central.

El trabajo se estructuró en dos objetivos:

- **Análisis de migración y construcción de dataset:** Investigación del proceso de actualización de la librería **Qiskit** (v0.46 a 1.0) de manera manual. Desarrollo de un *scraper* para la extracción de Pull Requests en repositorios de código abierto y generación de ejemplos sintéticos, consolidando un dataset híbrido para la validación del sistema en instancias futuras.
- **Entorno local para RAG:** Implementación de una arquitectura contenerizada con **Docker** (integrando **Ollama**, **Qdrant** y **N8N**) para automatizar los procesos de refactorización detectados en la etapa anterior.

2. Organigrama y Ubicación del Alumno

Dada la naturaleza del proyecto, no existe un organigrama jerárquico formal. La estructura de trabajo fue horizontal y se basó en la gestión de tareas a través de la coordinación presencial (metodología ágil Scrum) y mediante el uso de plataformas como Gmail y Whatsapp.

Mi rol dentro del proyecto fue el de **Desarrollador e Investigador**. Mis responsabilidades se centraron en:

- Diseñar, implementar y configurar el entorno de desarrollo local utilizando Docker y las tecnologías asociadas (Ollama, N8N, Qdrant) y garantizar el correcto funcionamiento de los mismos.
- Investigar y analizar las Pull Requests en proyectos de GitHub para identificar nuevos casos de migración en Qiskit, además de crear ejemplos sintéticos.
- Colaborar en el análisis de los resultados y la escritura del extended abstract

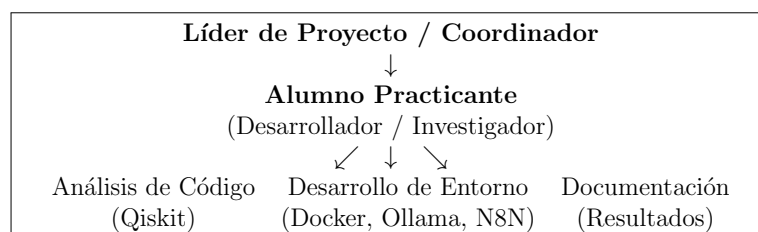


Figura 1: Esquema de la estructura de trabajo

3. Descripción de Tareas Realizadas (Orden cronológico)

3.1. Configuración del Entorno de Desarrollo con Docker

Una tarea fundamental fue la creación de un entorno de desarrollo robusto y reproducible mediante Docker. Esto permite aislar las dependencias y asegurar que todos los componentes funcionen de manera consistente.

Los pasos incluyeron la instalación y configuración de Docker en un sistema operativo basado en Ubuntu. Se resolvió un problema común de permisos para evitar la necesidad de ejecutar comandos de Docker con ‘sudo’.

Posteriormente, se procedió a levantar los servicios necesarios en contenedores separados:

- **Qdrant:** Base de datos vectorial para almacenar embeddings, esencial para sistemas RAG, configurada con persistencia de datos y mapeo de puertos para acceso externo.
- **N8N:** Herramienta de automatización de flujos de trabajo, utilizada para orquestar las interacciones lógicas entre el modelo de lenguaje y la base de datos vectorial.
- **Ollama:** Servicio para ejecutar modelos de lenguaje de gran tamaño (LLM) de forma local, permitiendo la integración de modelos como GPT-OSS-20b sin dependencia de la nube.

3.2. Análisis de Migración de Qiskit v0.46 a v1.0

La segunda parte del proyecto consistió en un trabajo de investigación sobre la actualización de la biblioteca de computación cuántica Qiskit. El objetivo era entender los cambios disruptivos entre la versión 0.46 y la 1.0 para poder migrar proyectos existentes.

Para ello, se creó un cuaderno en Google Colab que contiene 25 casos de migración de código en versión 1.0, a partir de los archivos de la versión 0.46 que los investigadores del LIFIA poseían con anterioridad. Para realizar la migración se utilizó en la medida de lo posible la "Migration Guide" de Qiskit y repositorios de StackOverflow. A continuación se adjunta un ejemplo de una migración completa del Ejemplo 16 provisto por el LIFIA.

Ejemplo 16

```
from qiskit import QuantumCircuit

qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)
qc.measure_all()

job = createBackendAndrunJob(qc)

result = job.result()
counts = result.get_counts()
```

MJigration guide

The qiskit.tools.events module and the progressbar() utility it exposed have been removed. This module's functionality was not widely used and is better covered by dedicated packages such as tqdm.

Figura 2: Ejemplo Migración a v1.0

Posteriormente, dado que los Investigadores requerían de más casos de Migración para mejorar la fiabilidad del experimento, se analizaron múltiples Pull Requests de repositorios públicos en GitHub.

Para obtener la Información de los mismos, se construyó un Web Scrapper en Google Colab, que descargaba los archivos modificados de Pull Request buscados como "Qiskit 1.0". Se identificaron y extrajeron los archivos modificados de 10 proyectos de código abierto para construir un dataset representativo. Los archivos de la versión 0.46 de Qiskit guardaban en la carpeta "base" y los posteriores (1.0) en la carpeta "head".

Posteriormente se creó un flujo de trabajo con scripts de shell para procesar los proyectos recolectados. Este flujo convertía los notebooks (.ipynb) a scripts de Python (.py), verificaba la integridad estructural (misma cantidad de archivos antes y después) y realizaba un análisis diferencial para contar la cantidad de líneas de código modificadas entre las versiones.

Pasos del Proceso de PostPorsesado:

1. **Ejecución de run_conversion.sh:** Este script automatiza la exportación del proyecto, creando una copia limpia donde transforma todos los notebooks Jupyter (.ipynb) en scripts Python (.py) y estandariza las extensiones de archivo para asegurar un entorno de ejecución puro.
2. **Ejecución de comparar.sh:** Este script compara las carpetas para verificar si contienen la misma cantidad de archivos y cuáles difieren.
3. **Ejecución de comparar_proyectos.sh:** Este script genera la tabla de resultados.

La ejecución del último script crea una tabla como la mostrada a continuación, a modo de ejemplo, correspondiente al Proyecto H

Archivo	+	-
runPoker.py	4	4
Board.py	48	28
PokerGame.py	20	16

Tabla 1: Resumen de cambios detectados.

Posteriormente, como no eran suficientes los ejemplos encontrados, para complementar los casos reales se crearon 46 ejemplos sintéticos con ayuda de inteligencia artificial en 3 archivos de Google Colab. Cada ejemplo incluía el código en Qiskit 0.46, su versión migrada a Qiskit 1.0 y una descripción de los cambios aplicados. A continuación se presenta un ejemplo de migración sintética:

```
from qiskit import QuantumCircuit, Aer, execute
import numpy as np

# Crear un circuito cuántico
qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0, 1)
qc.measure([0, 1], [0, 1])

# Obtener el backend del simulador desde el objeto global Aer
simulator = Aer.get_backend('qasm_simulator')

# Ejecutar el circuito
job = execute(qc, simulator, shots=1024)
result = job.result()
counts = result.get_counts(qc)
print("Cuentas:", counts)

/tmp/ipython-input-2625336369.py:11: DeprecationWarning: The 'qiskit.Aer' entry po
simulator = Aer.get_backend('qasm_simulator')
/tmp/ipython-input-2625336369.py:14: DeprecationWarning: The function ``qiskit.exe
job = execute(qc, simulator, shots=1024)
Cuentas: {'11': 518, '00': 506}
```

(a) Version 0.46

```
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator # 0 Aer.get_backend('qasm_simulator')
import numpy as np

# Crear un circuito cuántico
qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0, 1)
qc.measure([0, 1], [0, 1])

# Instanciar el simulador desde el paquete qiskit_aer
simulator = AerSimulator()

# Transpilar y ejecutar el circuito en dos pasos explícitos
transpiled_qc = transpile(qc, simulator)
job = simulator.run(transpiled_qc, shots=1024)
result = job.result()
counts = result.get_counts(transpiled_qc)Adicionales
print("Cuentas:", counts)

Cuentas: {'00': 480, '11': 544}
```

(b) Version 1.0

Figura 3: Migración Sintética

3.3. Implementación y Análisis del Sistema RAG

Una vez configurado el entorno, se procedió a la implementación y refinamiento del sistema de Generación Aumentada por Recuperación (RAG).

Para ello Se modificó el Template *Local Chatbot with Retrieval Augmented Generation (RAG)* de la web de N8N para adaptarlo a los requerimientos específicos del proyecto de migración de Qiskit. A continuación se procede a explicar los pasos realizados

Fases de Ejecución del Workflow

1. Configuración y Control (Globals / Start):

- Se establecen **variables globales** clave, incluyendo rutas de repositorios en GitHub, la versión de la documentación objetivo (ej. **0.46.0**), el modelo de lenguaje a evaluar (**ollama-gpt-oss-20b**), y el modo de operación (**Ingesta** o **Chatbot**).

2. Data Ingestion (Construcción de la Base de Conocimiento Semántico):

- El sistema accede a la documentación del proyecto en GitHub y la **filtra** por la versión objetivo.
- Los documentos se **descargan, preprocesan** (convertidos a texto plano) y se dividen en fragmentos (**chunks**).
- Los fragmentos se transforman en **vectores numéricos** (Embeddings) utilizando un modelo de bajo consumo (**bge-m3:567m**) alojado en Ollama.
- Finalmente, estos vectores se almacenan en la **Base de Datos Vectorial Qdrant**, creando el índice de conocimiento semántico (vector store) que consultará el chatbot.

3. Data Prossesing (Preparación de Casos de Prueba):

- Se obtiene un conjunto de **snippets de código Python** (casos de prueba) y los **prompts de usuario/sistema** predefinidos desde el repositorio.
- Los snippets se integran dinámicamente en los prompts de usuario, creando un **escenario de pregunta/respuesta** completo para cada caso de prueba.

4. Chatbot (Ejecución del Agente e Integración RAG):

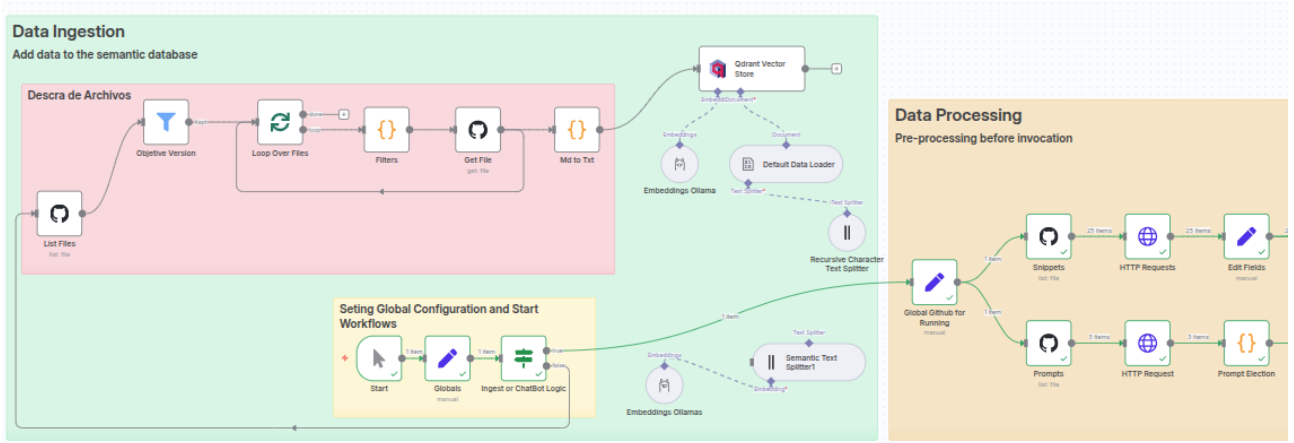
- Un **Agente de IA (AI Agent)** utiliza el modelo de lenguaje (**Google Gemini Chat Model, OpenAI Chat Model** o **GPTOSS-20B**) para generar una respuesta a la consulta.
- El agente está configurado para usar la base de datos **Qdrant como una herramienta (tool)**, permitiéndole recuperar información relevante para fundamentar su respuesta.

5. Files Upload (Gestión y Trazabilidad de Resultados):

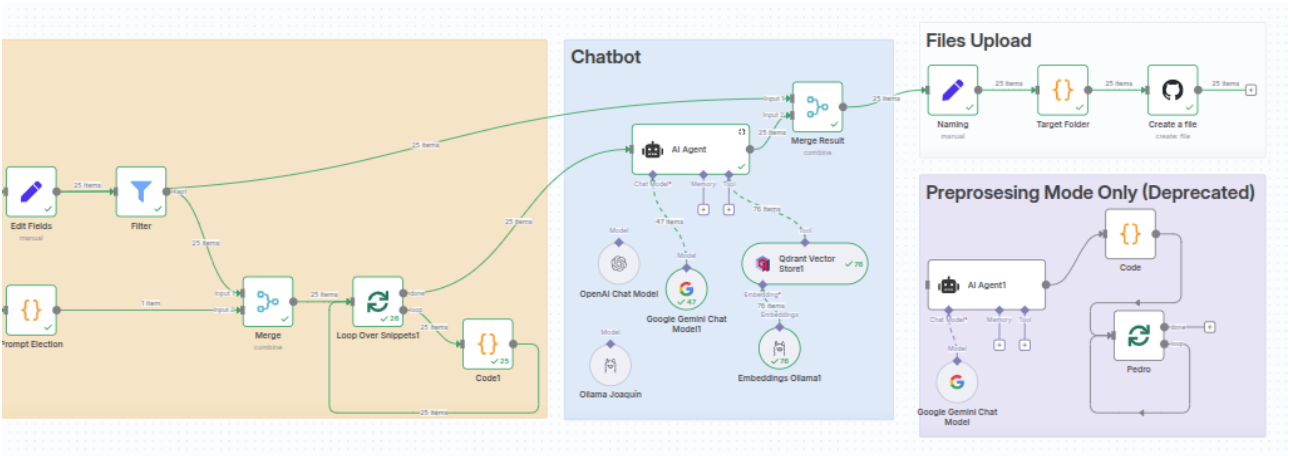
- Las respuestas generadas se **nombran, fechan** y se **almacenan** automáticamente en un directorio de resultados en GitHub, facilitando la auditoría y la evaluación comparativa del desempeño del chatbot.

El flujo de preprocesamiento (**Deprecated**) fue implementado inicialmente para estructurar y clasificar el código para el agente GPT-OSS-20B, cuya capacidad de razonamiento resultó limitada. Posteriormente, al cambiarse el agente RAG principal, que demostró parsear los *snippets* de forma autónoma y precisa, esta etapa se volvió innecesaria y se eliminó.

A continuacion se procede a mostrar el Workflow Completo



(a) Parte A



(b) Parte B

Figura 4: Implementación N8N: (a) Parte A, (b) Parte B

3.4. Verificación y optimización del RAG

Con el fin de mejorar los resultados obtenidos, se realizaron las siguientes acciones:

- **Procesamiento individual de ejemplos:** Se ejecutó el proceso de migración con el sistema RAG para cada uno de los 25 ejemplos de forma individual. El objetivo fue analizar si el RAG se ejecuta de manera correcta y consistente al aplicarse a cada caso en particular.
- **Evaluación de un enfoque de RAG menos restrictivo:** Atendiendo a las directrices del proyecto, se realizaron pruebas con el prompt de alta restricción, pero se realizó también un prompt de baja restricción (Sugerir acceso al RAG en vez de forzarlo) con el fin de realizar una comparación equitativa con trabajos de investigación previos. Se exploró además si modelos más avanzados (por ejemplo, Gemini Flash 2.5 Latest) pueden utilizar el RAG de forma más inteligente, detectando las librerías modificadas para realizar búsquedas específicas y manejar cambios de código complejos, dando como resultado mejoras sustanciales en las transformaciones respecto a GPT OSS.

3.5. Escritura del paper

La redacción del paper se realizó mediante un proceso estructurado que incluyó un diseño metodológico orientado a garantizar la reproducibilidad. Se efectuó un análisis objetivo de los resultados experimentales, asegurando que las conclusiones estuvieran sustentadas en evidencia técnica. Finalmente, se cuidó la coherencia narrativa, la estructura lógica y la revisión del texto para mantener la consistencia, el cumplimiento del formato académico y la claridad en todo el documento.

4. Tecnología Empleada

Durante la PPS, se utilizó un conjunto de tecnologías modernas y relevantes en el campo de la ingeniería de software y la inteligencia artificial.

Tecnologías Principales:

- **Lenguaje de Programación:** Python
- **Containerización:** Docker
- **Inteligencia Artificial:** Gemini, Ollama, Qdrant (Bases de Datos Vectoriales)
- **Automatización:** N8N
- **Computación Cuántica:** Qiskit
- **Control de Versiones:** Git, GitHub
- **Entornos de Desarrollo:** Google Colab, Visual Studio Code, N8N
- **Comunicación:** Whatsapp, Gmail

5. Problemas Identificados a lo largo de las Practicas

Durante el desarrollo del proyecto, se encontraron diversos desafíos técnicos que requirieron análisis y soluciones específicas:

- **Limitación en la Búsqueda de Proyectos de Referencia:** La búsqueda en GitHub de proyectos que hubieran migrado a Qiskit 1.0 resultó infructuosa a partir de la quinta página de resultados. Esto sugiere una escasa disponibilidad de pull requests públicas de migración bien documentadas y no borradas.
- **Dificultades en la Migración Manual de Ejemplos:** Durante la migración manual de 25 ejemplos, la guía de migración oficial no fue suficiente en todos los casos, ya que ciertos cambios estaban documentados en secciones externas. Si bien la consulta en plataformas como Stack Overflow permitió resolver la mayoría de estos problemas, un número reducido de casos no pudo ser transformado correctamente. Por lo que la release note sola puede que no sea suficiente para realizar un correcto cambio de versión.
- **Ineficacia de modulo en N8N:** Se implementó un divisor de texto semántico que no procesaba correctamente los documentos, omitiendo fragmentos de texto. Como solución, se revirtió al uso del “recursive text splitter” que venía por defecto en el Template de N8N, que funcionaba de manera fiable.

- **Incompatibilidad del LLM con Modelos Específicos:** Se detectaron problemas de compatibilidad con el LLM al intentar acceder con Ollama a modelos como Deepseek, los cuales no eran soportados por el modo de ejecución de agente.
- **Acceso Inconsistente al Sistema RAG:** El modelo Gpt_Oss_20b presentó fallos frecuentes al intentar acceder al sistema RAG. Para forzar la consulta, se diseñó un prompt de alta restricción que lograba entre una y tres llamadas, aunque a menudo de forma repetida y mal formulada. Sin embargo, al procesar los 25 ejemplos de forma simultánea, el rendimiento se degradó: el sistema falló incluso en casos que antes funcionaban y la tasa total de acceso al RAG disminuyó en un 66 %, a pesar del prompt restrictivo.

Además, dado que el tiempo fue insuficiente, quedo pendiente los siguientes aspectos:

- **Ampliación de la base de datos de comparación:** Se procesarán los ejemplos originales junto con los nuevos 56 ejemplos encontrados. Esto permitirá ampliar significativamente la base de datos para la comparación de resultados y obtener conclusiones más robustas.
- **Experimento futuro:** Se evaluará la capacidad del sistema RAG para migrar código a la versión más reciente de Qiskit, poniendo a prueba su adaptabilidad ante cambios y actualizaciones que no estén contemplados en los datos preentrenados.

6. Evaluación de la Formación Académica y Desempeño Profesional

A nivel personal, la experiencia fue sumamente enriquecedora. La realización de esta práctica ha sido mi primera experiencia laboral formal, lo cual ha reforzado mi interés por el desarrollo de software de alta complejidad y la inteligencia artificial, permitiéndome validar mis conocimientos en un entorno real.

6.1. Desempeño como Futuro Profesional

Esta práctica me permitió validar que poseo la capacidad de transformar requerimientos abstractos de investigación en soluciones tecnológicas concretas. Pude transicionar del rol de estudiante, al de un profesional que debe tomar decisiones de diseño, gestionar su tiempo de forma autónoma y justificar la elección y cambios de tecnologías críticas para el proyecto.

6.2. Fortalezas y Debilidades de la Formación Recibida

A partir de los desafíos enfrentados durante la implementación del sistema RAG y la migración de código, identifiqué los siguientes puntos respecto a mi formación de grado:

Fortalezas

- **Base Lógica:** La sólida formación provista por la universidad fue fundamental para comprender rápidamente conceptos complejos como los *embeddings* vectoriales, el sistema *RAG* y la lógica detrás de los circuitos cuánticos.
- **Capacidad de Abstracción:** La carrera ha fomentado una metodología de pensamiento que me permitió descomponer un problema grande (la migración masiva) en componentes manejables.

- **Autonomía en el Aprendizaje:** La exigencia académica me ha dotado de la disciplina necesaria para investigar y aprender tecnologías no vistas en la cursada en tiempos reducidos.

Debilidades detectadas

- **Herramientas de Infraestructura:** Se notó una carencia en contenidos prácticos sobre tecnologías de containerización (Docker) y despliegue, lo que requirió una curva de aprendizaje inicial pronunciada.
- **Tecnologías Emergentes:** Existe una brecha entre los contenidos académicos y el estado actual de la IA Generativa (LLMs), obligando a una actualización intensiva.

6.3. Adaptación Personal y Relaciones Interpersonales

Adaptación al Entorno y Grupo Humano

Mi adaptación personal a la entidad fue sumamente positiva. Como la modalidad de trabajo fue predominantemente presencial, la integración con el equipo fue inmediata gracias a la predisposición de los tutores. El ambiente laboral en el LIFIA se caracterizó por ser colaborativo y de respeto profesional, donde se valoraron mis aportes técnicos desde el primer día, haciéndome sentir un par más que un estudiante en formación.

Comunicación Interna

La dinámica de trabajo se caracterizó por una estructura horizontal, lo que facilitó un intercambio fluido de ideas. Al tratarse de un proyecto con componentes de investigación, las responsabilidades asumidas implicaron no solo el desarrollo técnico, sino también el reporte constante de hallazgos.

La comunicación fue principalmente **presencial**, complementada con el uso de plataformas de mensajería (WhatsApp) para la coordinación y herramientas de desarrollo (GitHub, Gmail) para el seguimiento técnico. Cabe destacar que, debido a la adopción de una metodología de trabajo ágil, no se realizaron informes de avance formales al Tutor Externo, realizándose el seguimiento de tareas mediante las reuniones de coordinación y la revisión de código.

7. Conclusión

La Práctica Profesional Supervisada ha cumplido con creces los objetivos planteados en el reglamento. Me ha permitido aplicar los conocimientos universitarios a problemas del mundo real, entender la dinámica del desarrollo de software en un entorno práctico y adquirir competencias técnicas en áreas de alta demanda como la containerización y la inteligencia artificial.

Esta experiencia ha sido un puente fundamental entre la formación académica y el mundo profesional, brindándome una perspectiva clara de los desafíos y las responsabilidades de un Ingeniero en Computación. He desarrollado habilidades no solo técnicas, sino también de resolución de problemas, autoaprendizaje y gestión de proyectos, las cuales considero esenciales para mi futuro profesional.

8. Bibliografía

- Lifa (2025). *Presentación del Paper: Qiskit_{CodeMigration}with_{LMs}.pdf* [Paper]. Recuperado de https://drive.google.com/file/d/1tb6nKUSlCCoNITZwu_UDjePHMKIYHq1Z/view?usp=drive_link
- Schwindt, I. A. (2025). *Notebook 1: Version Base (Qiskit 0.46)* [Código fuente]. Google Colab. Recuperado de https://colab.research.google.com/drive/1fgrbALr8240fe-HDYJFRS0mpFogxoXc_?usp=sharing
- Schwindt, I. A. (2025). *Notebook 2: Version Migrada (Qiskit 1.0)* [Código fuente]. Google Colab. Recuperado de <https://colab.research.google.com/drive/1l04w3wuSP6jn1h-Do5TlkyhsXWWVosiW?usp=sharing>
- Schwindt, I. A. (2025). *Notebook 3: Notas de Migración* [Dataset]. Google Colab. Recuperado de https://colab.research.google.com/drive/1qldVC2JYcV7Qk85aDTphBS0_RokHeAM2?usp=sharing
- Repositorio de Workflows (N8N): Carpeta con los flujos de automatización y configuración del entorno RAG. https://drive.google.com/drive/folders/1NquMzjBDI6t_vhZIr0vaaXaFhLRK7X_z
- Repositorio de Datos (Scraping): Scripts de extracción y dataset de Pull Requests de GitHub. <https://drive.google.com/drive/folders/1oJX4rPKzSLueskqFOXAXaDfVFwDccIjG>
- Docker Inc. (2024). *Docker Documentation: Containerize an application*. Recuperado de <https://docs.docker.com/>
- IBM Quantum. (2024). *Qiskit 1.0 Release Notes*. Qiskit Documentation. Recuperado de <https://docs.quantum.ibm.com/api/qiskit/release-notes/1.0>
- n8n. (2024). *n8n Workflow Automation Documentation*. Recuperado de <https://docs.n8n.io/>
- n8n. (2024). *Local Chatbot with Retrieval-Augmented Generation (RAG)*. n8n Workflows. Recuperado de <https://n8n.io/workflows/5148-local-chatbot-with-retrieval-augmented-generation-rag/>
- Ollama. (2024). *Ollama: Get up and running with large language models locally*. GitHub. Recuperado de <https://github.com/ollama/ollama>
- Qdrant. (2024). *Qdrant Documentation: Vector Database for AI*. Recuperado de <https://qdrant.tech/documentation/>
- Python Software Foundation. (2024). *Python 3.12.0 Documentation*. Recuperado de <https://docs.python.org/3/>
- Stack Overflow. (2025). *Qiskit Community Questions and Answers*. Recuperado de <https://stackoverflow.com/questions/tagged/qiskit>